MODUL 5 REAL-TIME OPERATING SYSTEM (RTOS)

Apriyudha (H1H024010)
Asisten: Arga Aryanta Indrafata
Tanggal Percobaan: 05/05/2026
TK244005-Praktikum Sistem Mikrokontroler
Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Praktikum ini bertujuan untuk memahami prinsip kerja Real-Time Operating System (RTOS) dan mengimplementasikan multitasking serta komunikasi antar task menggunakan FreeRTOS pada mikrokontroler Arduino. RTOS merupakan sistem operasi yang dirancang untuk memenuhi kebutuhan aplikasi real-time pada perangkat embedded, dengan kemampuan melakukan penjadwalan task secara konsisten. Dua percobaan dilakukan: pertama, implementasi multitasking dengan tiga task yang berjalan concurrently untuk mengontrol dua buah LED dan menampilkan counter pada Serial Monitor menggunakan fungsi xTaskCreate() dan vTaskStartScheduler(); kedua, implementasi komunikasi antar task menggunakan message queue (xQueueCreate, xQueueSend, xQueueReceive) untuk mengirim dan menerima data antar task. Hasil percobaan menunjukkan bahwa FreeRTOS berhasil diimplementasikan pada Arduino Uno, di mana ketiga task berjalan secara bergantian sesuai dengan prioritas dan waktu delay yang ditentukan oleh scheduler. Pada percobaan queue, data berhasil dikirim dari task pengirim ke task penerima melalui mekanisme queue tanpa terjadi race condition.

Kata kunci:*RTOS, FreeRTOS, Multitasking, Task Queue, Arduino, Scheduler*

1. Pendahuluan

Real-Time Operating System (RTOS) adalah sebuah Operating System (OS) yang digunakan untuk memenuhi kebutuhan aplikasi secara real-time pada embedded device yang memproses data secara langsung tanpa adanya penundaan (buffer). RTOS bekerja setiap saat dimana ia dibutuhkan saat itu juga, dengan kemampuan untuk melakukan kerja secara konsisten baik secara waktu yang dibutuhkan maupun secara task aplikasi yang mampu dikerjakan [1].

Pada sistem mikrokontroler konvensional, program dijalankan secara sequential dalam fungsi loop(), di mana setiap instruksi dieksekusi satu per satu secara berurutan. Pendekatan ini memiliki keterbatasan ketika aplikasi memerlukan penanganan multiple proses yang harus berjalan secara bersamaan. RTOS hadir sebagai solusi untuk mengatasi keterbatasan tersebut dengan menyediakan mekanisme multitasking, di mana beberapa task dapat berjalan secara concurrent melalui sistem penjadwalan yang dikelola oleh scheduler [2].

FreeRTOS adalah salah satu RTOS yang paling sering digunakan karena bersifat open source, gratis, dan mudah diimplementasikan pada berbagai platform mikrokontroler termasuk Arduino. FreeRTOS menyediakan fitur multitasking dengan preemptive scheduling, komunikasi antar task melalui queue, serta mekanisme sinkronisasi dengan semaphore dan mutex. Praktikum ini dilakukan untuk memberikan pemahaman praktis mengenai implementasi FreeRTOS pada Arduino, dengan fokus pada konsep multitasking dan komunikasi antar task menggunakan message queue [3].

2. Studi Pustaka

2.1 Konsep Real-Time Operating System (RTOS)

RTOS terdiri dari dua jenis utama: soft RTOS dan hard RTOS. Soft RTOS adalah sistem yang hampir selalu menyelesaikan task dengan waktu yang telah ditentukan. Pada soft RTOS, kemungkinan penyelesaian task melewati batas waktu (deadline) pelaksanaan masih bisa terjadi, dan kegagalan mencapai deadline tidak menimbulkan efek yang berbahaya bagi sistem, hanya berupa penurunan performa. Sedangkan hard RTOS merupakan sistem yang dipastikan selalu menyelesaikan task dalam waktu yang telah ditentukan. Hard RTOS selalu menyelesaikan task sebelum deadline, dan apabila terjadi kegagalan menyelesaikan task maka sistem akan mengalami efek berbahaya yang dapat merusak sistem secara keseluruhan [1].

Perbedaan utama antara sistem dengan RTOS dan non-RTOS terletak pada cara penjadwalan dan eksekusi task. Pada sistem non-RTOS, program berjalan dalam satu loop utama di mana setiap proses harus menunggu proses sebelumnya selesai. Sebaliknya, pada sistem RTOS, scheduler mengatur eksekusi multiple task dengan mekanisme time-slicing, sehingga setiap task mendapatkan bagian waktu CPU sesuai dengan prioritasnya [2].

2.2 FreeRTOS pada Arduino

FreeRTOS adalah RTOS open source yang paling populer untuk aplikasi embedded. Library Arduino_FreeRTOS menyediakan implementasi FreeRTOS yang kompatibel dengan board Arduino berbasis AVR seperti Arduino Uno, Leonardo, dan Mega. Kernel FreeRTOS sangat ringan, umumnya memerlukan kurang dari 10KB flash memory, sehingga cocok untuk mikrokontroler dengan sumber daya terbatas [3].

Fungsi-fungsi utama dalam FreeRTOS antara lain:

- xTaskCreate():Membuat task baru dengan parameter nama, ukuran stack, prioritas, dan handle.
- vTaskStartScheduler():Memulai scheduler yang mengatur penjadwalan task.
- vTaskDelay():Menunda eksekusi task selama jumlah tick yang ditentukan.
- vTaskDelayUntil():Menunda eksekusi task sampai waktu absolut tertentu, berguna untuk task periodik.

2.3 Task Queue dan Inter-Task Communication

Queue adalah mekanisme komunikasi antar task dalam FreeRTOS yang memungkinkan pengiriman data dari satu task ke task lain. Queue bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan pertama keluar. Queue memiliki kapasitas tetap yang ditentukan saat pembuatan, dan operasi pengiriman serta penerimaan data bersifat blocking [4].

Fungsi-fungsi utama untuk operasi queue adalah xQueueCreate() untuk membuat queue, xQueueSend() untuk mengirim data ke queue, dan xQueueReceive() untuk menerima data dari queue. Parameter portMAX_DELAY digunakan untuk menunggu tanpa batas waktu jika queue penuh (pengiriman) atau kosong (penerimaan). Mekanisme queue ini mencegah terjadinya race condition karena hanya satu task yang dapat mengakses queue pada satu waktu [5].

Tabel berikut menunjukkan perbedaan antara sistem non-RTOS dan RTOS:

Tabel 2-1 Perbedaan Sistem Non-RTOS dan RTOS

Karakteristik	Non-RTOS	RTOS
Metode Eksekusi	Berjalan secara berurutan (loop)	Berjalan secara bersamaan (task)
Penjadwalan	Tidak menggunakan scheduler	Menggunakan preemptive scheduler
Multitasking	Tidak mendukung multitasking	Mendukung banyak task sekaligus
Berbagi Resource	Diatur manual	Menggunakan Queue, Semaphore, dan Mutex
Waktu Respons	Tidak pasti (non-deterministik)	Pasti dan terukur (deterministik)
Tingkat Kompleksitas	Lebih sederhana	Lebih kompleks

3. Metodologi

3.1 Alat dan Bahan

Percobaan ini menggunakan perangkat keras dan perangkat lunak sebagai berikut:

Tabel 3-1 Daftar Alat dan Bahan

No	Komponen	Keterangan
1	Arduino Uno	Mikrokontroler utama
2	LED Merah (2 buah)	Indikator output task
3	Resistor 220 Ohm	Pembatas arus LED sebanyak 2 buah
4	Breadboard	Media perakitan rangkaian
5	Kabel Jumper	Penghubung antar komponen
6	Arduino IDE	Software untuk pemrograman Arduino
7	Library Arduino_FreeRTOS	Library RTOS yang digunakan pada Arduino

3.2 Konfigurasi Pin

Konfigurasi pin untuk percobaan multitasking adalah sebagai berikut:

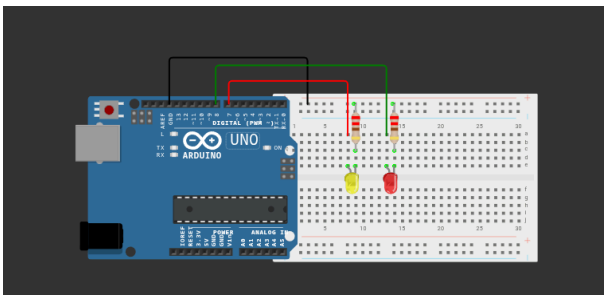
Tabel 3-2 Konfigurasi Pin

Komponen	Pin Arduino	Keterangan

LED Merah 1	Pin 7	Output digital untuk TaskBlink1
LED Merah 2	Pin 8	Output digital untuk TaskBlink2
Resistor 1	Seri dengan LED 1	Pembatas arus menuju GND
Resistor 2	Seri dengan LED 2	Pembatas arus menuju GND
GND	GND	Ground bersama

3.3 Skema Rangkaian dan Langkah Percobaan

3.3.1 Percobaan 1: Multitasking



Gambar 3-1 Skema Rangkaian Percobaan 1

1) Persiapkan alat dan bahan sesuai tabel 3-1

2) Program Percobaan

```
C/C++
#include <Arduino_FreeRTOS.h>

void TaskBlink1(void *pvParameters);
void TaskBlink2(void *pvParameters);
void Taskprint(void *pvParameters);

void setup() {
  // initialize serial communication
at 9600 bits per second:
  Serial.begin(9600);

  xTaskCreate(TaskBlink1, "task1",
128, NULL, 1, NULL);
  xTaskCreate(TaskBlink2, "task2",
128, NULL, 1, NULL);
  xTaskCreate(Taskprint, "task3",
128, NULL, 1, NULL);

  vTaskStartScheduler();
}

void loop() {
}

void TaskBlink1(void *pvParameters) {
  pinMode(8, OUTPUT);

  while (1) {
    Serial.println("Task1");

    digitalWrite(8, HIGH);
    vTaskDelay(200 /
portTICK_PERIOD_MS);

    digitalWrite(8, LOW);
    vTaskDelay(200 /
portTICK_PERIOD_MS);
  }
}

void TaskBlink2(void *pvParameters) {
  pinMode(7, OUTPUT);

  while (1) {
    Serial.println("Task2");

    digitalWrite(7, HIGH);
    vTaskDelay(300 /
portTICK_PERIOD_MS);

    digitalWrite(7, LOW);
    vTaskDelay(300 /
portTICK_PERIOD_MS);
  }
}

void Taskprint(void *pvParameters) {
  int counter = 0;

  while (1) {
    counter++;
    Serial.println(counter);

    vTaskDelay(500 /
portTICK_PERIOD_MS);
  }
}
```

3) Simpan sketch dengan nama file modul5_percobaan1
4) Konfigurasi rangkaian sesuai gambar 3-1
5) Berikutnya compile dan upload program ke dalam Arduino board

3.3.2 Percobaan 2: Komunikasi Task dengan Queue

1) Persiapkan alat dan bahan sesuai tabel 3-1

2) Program Percobaan

```
C/C++
#include <Arduino_FreeRTOS.h>
#include <queue.h>

struct readings {
  int temp;
  int h;
};

QueueHandle_t my_queue;

void setup() {
  Serial.begin(9600);

  my_queue = xQueueCreate(1,
sizeof(struct readings));

  xTaskCreate(read_data, "read
sensors", 128, NULL, 0, NULL);
  xTaskCreate(display, "display",
128, NULL, 0, NULL);
}

void loop() {
}

/*
 * Blink task.
 * See Blink_AnalogRead example.
 */

void read_data(void *pvParameters) {
  struct readings x;

  for (;;) {
    x.temp = 54;
    x.h = 30;

    xQueueSend(my_queue, &x,
portMAX_DELAY);

    vTaskDelay(100);
  }
}

void display(void *pvParameters) {
  struct readings x;

  for (;;) {
    if (xQueueReceive(my_queue, &x,
portMAX_DELAY) == pdPASS) {

      Serial.print("temp = ");
      Serial.println(x.temp);
      Serial.print("humidity = ");
      Serial.println(x.h);
    }
  }
}
```

3) Simpan sketch dengan nama file modul5_percobaan2
4) Berikutnya compile dan upload program ke dalam Arduino board

4. Hasil dan Analisis

4.1 Hasil Percobaan 1: Multitasking

Pada percobaan pertama, tiga task berhasil dibuat dan dijalankan secara concurrent menggunakan FreeRTOS. TaskBlink1 mengontrol LED pada pin 8 dengan interval delay 200 ms, TaskBlink2 mengontrol LED pada pin 10 dengan interval delay 300 ms, dan TaskPrint menampilkan nilai counter yang terus bertambah setiap 500 ms pada Serial Monitor.

Hasil pengamatan menunjukkan bahwa ketiga task tidak berjalan secara bersamaan (parallel), melainkan secara bergantian (concurrent) melalui mekanisme time-slicing yang dikelola oleh scheduler. Scheduler FreeRTOS menggunakan algoritma preemptive priority-based scheduling, di mana task dengan prioritas lebih tinggi akan dieksekusi terlebih dahulu. Karena ketiga task diberi prioritas yang sama (1), scheduler memberikan waktu eksekusi yang sama rata kepada setiap task [1].

Tabel 4-1 Hasil Pengamatan Percobaan 1

Task	Pin/Output	Delay	Output Serial
TaskBlink1	Pin 7 (LED)	200 ms	"Task1"
TaskBlink2	Pin 8 (LED)	300 ms	"Task2"
TaskPrint	Serial Monitor	500 ms	Counter (1, 2, 3, ...)

4.2 Hasil Percobaan 2: Komunikasi Task dengan Queue

Pada percobaan kedua, komunikasi antar task berhasil diimplementasikan menggunakan message queue. Task read_data mengirimkan data struct berisi nilai temperature dan humidity ke queue menggunakan fungsi xQueueSend(). Task display menerima data dari queue menggunakan fungsi xQueueReceive() dan menampilkannya pada Serial Monitor.

Hasil pengamatan pada Serial Monitor menunjukkan bahwa data berhasil dikirim dan diterima dengan benar. Nilai temperature (54) dan humidity (30) ditampilkan secara terus-menerus, menunjukkan bahwa mekanisme queue berfungsi dengan baik. Penggunaan portMAX_DELAY pada fungsi xQueueSend dan xQueueReceive memastikan bahwa task akan menunggu sampai operasi queue berhasil, sehingga tidak ada data yang hilang [4].

Tabel 4-2 Hasil Pengamatan Percobaan 2

Task	Fungsi	Output Serial Monitor
read_data	Mengirim data ke queue	temp = 54, humidity = 30
display	Menerima data dari queue	Data dikirim ke queue

4.3 Analisis

Berdasarkan hasil kedua percobaan, dapat dianalisis bahwa FreeRTOS berhasil diimplementasikan pada Arduino Uno untuk aplikasi multitasking dan komunikasi antar task. Scheduler FreeRTOS mengatur eksekusi task dengan efektif, memastikan setiap task mendapatkan waktu CPU sesuai dengan prioritasnya. Penggunaan vTaskDelay() yang menggantikan fungsi delay() standar Arduino memungkinkan scheduler untuk beralih ke task lain selama task yang sedang berjalan berada dalam kondisi blocked [2].

Pada percobaan queue, mekanisme message passing antara task berhasil mencegah terjadinya race condition. Race condition adalah kondisi di mana dua atau lebih task mengakses data bersamaan secara tidak terkoordinasi, yang dapat menyebabkan data corruption. Dengan menggunakan queue, hanya satu task yang dapat mengakses data pada satu waktu, sehingga integritas data terjaga [5].

Perbedaan antara loop() pada Arduino biasa dengan sistem yang menggunakan RTOS terletak pada cara eksekusi program. Pada Arduino biasa, loop() dieksekusi secara sequential dan tidak ada mekanisme untuk menjalankan multiple proses secara bersamaan. Sebaliknya, pada sistem RTOS, fungsi loop() dibiarkan kosong karena seluruh eksekusi program dijalankan dalam task-task yang terpisah dan dikelola oleh scheduler. Hal ini memungkinkan pengembangan aplikasi yang lebih kompleks dengan multiple proses yang berjalan secara concurrent [3].

5. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa:

- FreeRTOS berhasil diimplementasikan pada Arduino Uno untuk menjalankan multiple task secara concurrent melalui mekanisme multitasking.
- Tiga task (TaskBlink1, TaskBlink2, dan TaskPrint) berhasil berjalan secara bergantian sesuai dengan penjadwalan yang dikelola oleh scheduler.
- Komunikasi antar task berhasil diimplementasikan menggunakan message queue (xQueueCreate, xQueueSend, xQueueReceive) tanpa terjadi race condition.
- Pemahaman konsep RTOS dan FreeRTOS merupakan keterampilan penting dalam pengembangan sistem embedded yang memerlukan penanganan multiple proses secara real-time.

Daftar Pustaka

[1] Sakthivel V, Sreeja P. (2024). Synchronization Techniques in Real-Time Operating Systems: Implementation and Evaluation on Arduino with FreeRTOS. International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), 13(2). https://doi.org/10.17148/IJARCCE.2024.13246

[2] Biondi, A., Buttazzo, G. (2016). ARTE: Arduino Real-Time Extension for Programming Arduino Boards. Proceedings of the 31st Annual ACM Symposium on Applied Computing. https://doi.org/10.1145/2851613.2851672

[3] Cheng, Z., Li, Y., West, R. (2015). Qduino: A Multithreaded Arduino System for Embedded Computing. Proceedings of the 36th IEEE Real-Time Systems Symposium (RTSS 2015), San Antonio, Texas.

[4] Kondaveeti, H. K., Kumaravelu, N. K., Vanambathina, S. D., Mathe, S. E., & Vappangi, S. (2021). A Systematic Literature Review on Prototyping with Arduino: Applications, Challenges, Advantages, and Limitations. Computer Science Review, 40, 100364. https://doi.org/10.1016/j.cosrev.2021.100364

[5] Tselegkaridis, S., Sapounidis, T., & Papakostas, D. (2024). Learning Circuits and Coding with Arduino Board in Higher Education Using Tangible and Graphical User Interfaces. Information, 15(5), 245. https://doi.org/10.3390/info15050245